

Apostila: carga de dados fiscais com BULK INSERT

Sumario

1. Objetivos e arquitetura do processo
 2. Arquivos utilizados
 3. Pre-requisitos do SQL Server
 4. Etapa 1 - preparacao e construcao das tabelas de staging
 5. Etapa 1 - carga bruta para staging
 6. Validacao das tabelas de staging
 7. Etapa 2 - consolidacao nas tabelas fiscais finais
 8. Passo a passo da consolidacao de cada tabela
 9. Conferencia dos resultados
 10. Tratamento de erros e reprocessamento
 11. Boas praticas e problemas comuns
-

1. Objetivos e arquitetura do processo

Esta apostila apresenta um fluxo completo de ingestao de arquivos CSV no SQL Server. O processo possui duas etapas bem separadas:

```
CSV -> BULK INSERT -> tabelas staging -> validacao/conversao -> tabelas fiscal
```

Etapa 1 - staging

Os arquivos sao carregados primeiro em tabelas do schema **staging**. Os campos que vieram dos arquivos sao armazenados como **NVARCHAR**. Assim, um valor como **ABC** em uma coluna que deveria ser numerica nao interrompe a ingestao. O problema sera identificado na etapa de validacao.

Etapa 2 - tabelas finais

Os dados são convertidos com `TRY_CONVERT` e inseridos nas tabelas do schema `fiscal`. Nessa etapa passam a valer as regras de negócio e integridade definidas no arquivo `criar_tabelas_fiscais_finais.sql`: chaves primárias, chaves estrangeiras, valores não negativos e domínios de situação.

Essa separação preserva o dado bruto e permite auditar, corrigir e reprocessar arquivos sem perder a origem.

2. Arquivos utilizados

Todos os arquivos desta apostila estão no diretório `exemplos`.

Arquivo	Tabela staging	Quantidade de campos
<code>staging_nfe_20.csv</code>	<code>staging.stg_nfe_bulk</code>	6
<code>staging_item_nfe_20.csv</code>	<code>staging.stg_item_nfe_bulk</code>	4
<code>staging_nfce_20.csv</code>	<code>staging.stg_nfce_bulk</code>	5
<code>staging_item_nfce_20.csv</code>	<code>staging.stg_item_nfce_bulk</code>	4
<code>staging_cte_20.csv</code>	<code>staging.stg_cte_bulk</code>	5
<code>staging_efd_registro_20.csv</code>	<code>staging.stg_efd_registro_bulk</code>	4

Os CSVs usam:

- `;` como separador de campos;
- UTF-8, indicado por `CODEPAGE = '65001'`;
- cabeçalho na primeira linha;
- fim de linha LF, carregado com `ROWTERMINATOR = '0x0A'`;
- datas no formato `YYYY-MM-DD`, exceto `periodo_apuracao`, que usa `MM/YYYY`;
- valores monetários com ponto decimal.

O arquivo `nfe_sample.csv` tem outro layout e pertence ao exemplo antigo de NF-e. Ele não deve ser carregado nas tabelas `stg_*` desta apostila.

3. Pre-requisitos do SQL Server

3.1 Caminho dos arquivos

O **BULK INSERT** é executado pelo serviço do SQL Server. Portanto, o caminho informado deve ser acessível pela conta do serviço, e não somente pelo usuário que abriu o SSMS ou o VS Code.

Copie os seis CSVs para um diretório acessível, por exemplo:

```
C:\dados\staging\
```

Em servidor remoto, prefira um compartilhamento UNC com permissão de leitura:

```
\\servidor\dados\staging\
```

3.2 Permissão de carga

O usuário precisa ter permissão **ADMINISTER BULK OPERATIONS** ou pertencer a uma role adequada, como **bulkadmin**, conforme a política do ambiente.

3.3 Ordem de execução

Execute os scripts nesta ordem:

1. **staging_bulk_tabelas.sql** ou a parte de criação de staging de **criar_stg_nfe_bulk.sql**;
2. os comandos de **BULK INSERT** desta apostila;
3. **criar_tabelas_fiscais_finais.sql**;
4. os comandos de consolidação da Etapa 2.

Em um ambiente limpo, também é possível usar o **criar_stg_nfe_bulk.sql**, que recria as tabelas e carrega os seis arquivos. Ajuste primeiro a variável **@pasta_csv** nesse script.

Atencao: o comando `CREATE SCHEMA IF NOT EXISTS` presente em `staging_bulk_tabelas.sql` nao e sintaxe valida do SQL Server. Use o bloco `IF NOT EXISTS ... EXEC('CREATE SCHEMA staging')` existente em `criar_stg_nfe_bulk.sql` ou crie o schema manualmente.

4. Etapa 1 - preparacao e construcao das tabelas de staging

Antes de importar qualquer arquivo, crie o schema `staging` e as tabelas que receberao os dados brutos. A estrutura deve corresponder ao cabecalho e a ordem das colunas de cada CSV.

4.1 Criar o schema

`CREATE SCHEMA IF NOT EXISTS` nao existe no SQL Server. Use uma verificacao com `sys.schemas`:

```
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'staging')
BEGIN
    EXEC('CREATE SCHEMA staging');
END;
GO
```

4.2 Criar as tabelas de staging

As colunas de origem sao textuais de proposito. A staging deve receber o conteudo bruto sem tentar transformar datas, identificadores ou valores monetarios. A coluna `imported_dt` e uma coluna de controle e nao faz parte dos CSVs.

```
CREATE TABLE staging.stg_nfe_bulk (
    chave_acesso          NVARCHAR(44) NOT NULL,
    id_contribuinte_emitente NVARCHAR(20) NULL,
    id_contribuinte_destinatario NVARCHAR(20) NULL,
    data_emissao           NVARCHAR(10) NULL,
    valor_total            NVARCHAR(20) NULL,
    situacao               NVARCHAR(20) NULL
);
GO
```

```

CREATE TABLE staging.stg_item_nfe_bulk (
    id_nfe          NVARCHAR(20) NULL,
    ncm             NVARCHAR(8)  NULL,
    cfop            NVARCHAR(4)  NULL,
    valor_item      NVARCHAR(20) NULL
);
GO

CREATE TABLE staging.stg_nfce_bulk (
    chave_acesso      NVARCHAR(44) NOT NULL,
    id_contribuinte_emitente NVARCHAR(20) NULL,
    data_emissao       NVARCHAR(10) NULL,
    valor_total        NVARCHAR(20) NULL,
    situacao           NVARCHAR(20) NULL
);
GO

CREATE TABLE staging.stg_item_nfce_bulk (
    id_nfce          NVARCHAR(20) NULL,
    ncm             NVARCHAR(8)  NULL,
    cfop            NVARCHAR(4)  NULL,
    valor_item      NVARCHAR(20) NULL
);
GO

CREATE TABLE staging.stg_cte_bulk (
    chave_acesso      NVARCHAR(44) NOT NULL,
    id_contribuinte_emitente NVARCHAR(20) NULL,
    id_nfe_vinculada   NVARCHAR(20) NULL,
    valor_frete        NVARCHAR(20) NULL,
    situacao           NVARCHAR(20) NULL
);
GO

CREATE TABLE staging.stg_efd_registro_bulk (
    id_contribuinte      NVARCHAR(20) NULL,
    tipo_registro        NVARCHAR(10) NULL,
    periodo_apuracao     NVARCHAR(7)  NULL,
    id_nfe_referenciada  NVARCHAR(20) NULL
);
GO

```

4.3 Coluna de controle da importacao

Como `imported_dt` nao existe no arquivo, adicione-a depois dos `BULK INSERT`. O valor padrao sera preenchido pelo SQL Server para as linhas carregadas:

```

ALTER TABLE staging.stg_nfe_bulk
ADD imported_dt DATETIME2 NOT NULL
    CONSTRAINT DF_stg_nfe_bulk_imported_dt DEFAULT SYSUTCDATETIME();
GO

```

```

ALTER TABLE staging.stg_item_nfe_bulk
ADD imported_dt DATETIME2 NOT NULL
    CONSTRAINT DF_stg_item_nfe_bulk_imported_dt DEFAULT SYSUTCDATETIME();
GO

ALTER TABLE staging.stg_nfce_bulk
ADD imported_dt DATETIME2 NOT NULL
    CONSTRAINT DF_stg_nfce_bulk_imported_dt DEFAULT SYSUTCDATETIME();
GO

ALTER TABLE staging.stg_item_nfce_bulk
ADD imported_dt DATETIME2 NOT NULL
    CONSTRAINT DF_stg_item_nfce_bulk_imported_dt DEFAULT SYSUTCDATETIME();
GO

ALTER TABLE staging.stg_cte_bulk
ADD imported_dt DATETIME2 NOT NULL
    CONSTRAINT DF_stg_cte_bulk_imported_dt DEFAULT SYSUTCDATETIME();
GO

ALTER TABLE staging.stg_efd_registro_bulk
ADD imported_dt DATETIME2 NOT NULL
    CONSTRAINT DF_stg_efd_registro_bulk_imported_dt DEFAULT SYSUTCDATETIME();
GO

```

O arquivo `staging_bulk_tabelas.sql` apresenta a mesma estrutura com `imported_dt` declarado dentro do `CREATE TABLE`. Para cargas com `BULK INSERT` sem arquivo de formato, a estratégia usada nesta apostila é criar primeiro apenas as colunas presentes no CSV, carregar os dados e adicionar a coluna de controle depois. Isso evita que uma coluna que não existe no arquivo seja interpretada como parte do registro.

4.4 Recriar as tabelas em um ambiente de testes

Se o exercício for repetido, remova as tabelas dependentes antes das tabelas pai ou use o `criar_stg_nfe_bulk.sql`, que já recria as seis tabelas. Em uma base sem dependências, o padrão é:

```

IF OBJECT_ID('staging.stg_efd_registro_bulk', 'U') IS NOT NULL DROP TABLE
staging.stg_efd_registro_bulk;
IF OBJECT_ID('staging.stg_cte_bulk', 'U') IS NOT NULL DROP TABLE
staging.stg_cte_bulk;
IF OBJECT_ID('staging.stg_item_nfce_bulk', 'U') IS NOT NULL DROP TABLE
staging.stg_item_nfce_bulk;
IF OBJECT_ID('staging.stg_nfce_bulk', 'U') IS NOT NULL DROP TABLE
staging.stg_nfce_bulk;
IF OBJECT_ID('staging.stg_item_nfe_bulk', 'U') IS NOT NULL DROP TABLE

```

```
staging.stg_item_nfe_bulk;  
IF OBJECT_ID('staging.stg_nfe_bulk', 'U') IS NOT NULL DROP TABLE  
staging.stg_nfe_bulk;  
GO
```

Depois execute novamente os **CREATE TABLE** da secao 4.2.

5. Etapa 1 - carga bruta para staging

4.1 Por que usar SQL dinamico?

O caminho da clausula **FROM** do **BULK INSERT** nao aceita uma variavel diretamente em um comando estatico. Por isso, o caminho e o nome do arquivo sao concatenados em uma string e executados com **sys.sp_executesql**.

O trecho abaixo e um modelo para todos os arquivos:

```
DECLARE @pasta_csv NVARCHAR(260) = N'C:\dados\staging\';  
DECLARE @sql NVARCHAR(MAX);  
  
SET @sql = N'BULK INSERT staging.stg_nfe_bulk  
FROM ''' + @pasta_csv + N'staging_nfe_20.csv''  
WITH (  
    FIRSTROW = 2,  
    FIELDTERMINATOR = ';',  
    ROWTERMINATOR = '0x0A',  
    CODEPAGE = '65001',  
    TABLOCK  
);  
  
EXEC sys.sp_executesql @sql;
```

Explicacao dos parametros:

Parametro	Funcao
FIRSTROW = 2	ignora o cabecalho
FIELDTERMINATOR = ';' 	informa que as colunas sao separadas por ponto e virgula
ROWTERMINATOR = '0x0A'	informa que as linhas terminam com LF

Parametro	Funcao
<code>CODEPAGE = '65001'</code>	interpreta o arquivo como UTF-8
<code>TABLOCK</code>	aplica bloqueio de tabela e pode melhorar o desempenho

4.2 Carga dos seis arquivos

O bloco a seguir carrega cada arquivo na tabela correspondente. As tabelas precisam existir antes da execucao.

```

DECLARE @pasta_csv NVARCHAR(260) = N'C:\dados\staging\';
DECLARE @sql NVARCHAR(MAX);

SET @sql = N'BULK INSERT staging.stg_nfe_bulk
FROM ''' + @pasta_csv + N'staging_nfe_20.csv''
WITH (FIRSTROW = 2, FIELDTERMINATOR = ';' , ROWTERMINATOR = ''0x0A'', CODEPAGE =
''65001'', TABLOCK);';
EXEC sys.sp_executesql @sql;

SET @sql = N'BULK INSERT staging.stg_item_nfe_bulk
FROM ''' + @pasta_csv + N'staging_item_nfe_20.csv''
WITH (FIRSTROW = 2, FIELDTERMINATOR = ';' , ROWTERMINATOR = ''0x0A'', CODEPAGE =
''65001'', TABLOCK);';
EXEC sys.sp_executesql @sql;

SET @sql = N'BULK INSERT staging.stg_nfce_bulk
FROM ''' + @pasta_csv + N'staging_nfce_20.csv''
WITH (FIRSTROW = 2, FIELDTERMINATOR = ';' , ROWTERMINATOR = ''0x0A'', CODEPAGE =
''65001'', TABLOCK);';
EXEC sys.sp_executesql @sql;

SET @sql = N'BULK INSERT staging.stg_item_nfce_bulk
FROM ''' + @pasta_csv + N'staging_item_nfce_20.csv''
WITH (FIRSTROW = 2, FIELDTERMINATOR = ';' , ROWTERMINATOR = ''0x0A'', CODEPAGE =
''65001'', TABLOCK);';
EXEC sys.sp_executesql @sql;

SET @sql = N'BULK INSERT staging.stg_cte_bulk
FROM ''' + @pasta_csv + N'staging_cte_20.csv''
WITH (FIRSTROW = 2, FIELDTERMINATOR = ';' , ROWTERMINATOR = ''0x0A'', CODEPAGE =
''65001'', TABLOCK);';
EXEC sys.sp_executesql @sql;

SET @sql = N'BULK INSERT staging.stg_efd_registro_bulk
FROM ''' + @pasta_csv + N'staging_efd_registro_20.csv''
WITH (FIRSTROW = 2, FIELDTERMINATOR = ';' , ROWTERMINATOR = ''0x0A'', CODEPAGE =
''65001'', TABLOCK);';
EXEC sys.sp_executesql @sql;

```


4.3 Carga repetivel

O script `criar_stg_nfe_bulk.sql` adiciona `imported_dt` depois da carga. Isso evita que a coluna de controle seja interpretada como um campo do CSV. Para executar o processo novamente, recrie as tabelas ou use `TRUNCATE TABLE` antes de carregar, desde que nao existam dependencias que impeçam o truncate.

Nao execute o mesmo `BULK INSERT` duas vezes sobre uma tabela ja carregada sem limpar os dados, pois os registros serao duplicados.

6. Validacao das tabelas de staging

5.1 Conferir quantidade de registros

Cada arquivo de exemplo possui 20 registros:

```
SELECT 'NFE' AS origem, COUNT(*) AS total FROM staging.stg_nfe_bulk
UNION ALL
SELECT 'ITEM_NFE', COUNT(*) FROM staging.stg_item_nfe_bulk
UNION ALL
SELECT 'NFCE', COUNT(*) FROM staging.stg_nfce_bulk
UNION ALL
SELECT 'ITEM_NFCE', COUNT(*) FROM staging.stg_item_nfce_bulk
UNION ALL
SELECT 'CTE', COUNT(*) FROM staging.stg_cte_bulk
UNION ALL
SELECT 'EFD_REGISTRO', COUNT(*) FROM staging.stg_efd_registro_bulk;
```

Resultado esperado: 20 linhas para cada tabela.

5.2 Procurar valores que nao convertem

Antes da consolidacao, teste os campos que serao convertidos:

```
SELECT *
FROM staging.stg_nfe_bulk
WHERE TRY_CONVERT(BIGINT, id_contribuinte_emitente) IS NULL
      OR TRY_CONVERT(BIGINT, id_contribuinte_destinatario) IS NULL
```

```
OR TRY_CONVERT(DATE, data_emissao) IS NULL  
OR TRY_CONVERT(DECIMAL(16,2), valor_total) IS NULL;
```

O mesmo padrao pode ser usado para `valor_item`, `valor_frete`, `id_nfe`, `id_nfce` e `id_nfe_referenciada`.

5.3 Procurar situacoes fora do dominio

```
SELECT DISTINCT situacao FROM staging.stg_nfe_bulk;  
SELECT DISTINCT situacao FROM staging.stg_nfce_bulk;  
SELECT DISTINCT situacao FROM staging.stg_cte_bulk;
```

As situacoes devem corresponder aos `CHECK` das tabelas finais.

7. Etapa 2 - consolidacao nas tabelas fiscais finais

6.1 Criar o modelo definitivo

Execute `criar_tabelas_fiscais_finais.sql`. Ele cria:

- `fiscal.CONTRIBUINTE`;
- `fiscal.NFE`;
- `fiscal.ITEM_NFE`;
- `fiscal.NFCE`;
- `fiscal.ITEM_NFCE`;
- `fiscal.CTE`;
- `fiscal.EFD_REGISTRO`.

As tabelas finais usam `IDENTITY`. Portanto, seus identificadores sao gerados pelo SQL Server e nao devem ser confundidos automaticamente com todos os identificadores textuais existentes nos CSVs.

6.2 Cadastrar contribuintes antes dos documentos

As tabelas **NFE**, **NFCE**, **CTE** e **EFD_REGISTRO** possuem chaves estrangeiras para **fiscal.CONTRIBUINTE**. O cadastro deve existir antes da carga dos documentos.

Para este conjunto didatico, os CSVs usam IDs de contribuinte de **1** a **10**. Em um banco vazio, um cadastro minimo pode ser criado assim:

```
INSERT INTO fiscal.CONTRIBUINTE (cnpj, razao_social, uf)
VALUES
('10000000000001', 'Contribuinte 01', 'PB'),
('10000000000002', 'Contribuinte 02', 'PB'),
('10000000000003', 'Contribuinte 03', 'PB'),
('10000000000004', 'Contribuinte 04', 'PB'),
('10000000000005', 'Contribuinte 05', 'PB'),
('10000000000006', 'Contribuinte 06', 'PB'),
('10000000000007', 'Contribuinte 07', 'PB'),
('10000000000008', 'Contribuinte 08', 'PB'),
('10000000000009', 'Contribuinte 09', 'PB'),
('10000000000010', 'Contribuinte 10', 'PB');
```

Em um ambiente real, nao se deve presumir que a identidade 1 corresponde ao contribuinte 1 do arquivo. Deve-se localizar o contribuinte por uma chave natural, como CNPJ, ou manter uma tabela de mapeamento entre o identificador de origem e o identificador interno.

8. Passo a passo da consolidacao de cada tabela

Os comandos seguintes assumem que as tabelas de staging foram validadas e que o cadastro de contribuintes esta preenchido.

7.1 NFE

Campos de origem e destino

Staging	Tabela final	Conversao
chave_acesso	fiscal.NFE.chave_acesso	texto de 44 digitos
id_contribuinte_emitente	id_contribuinte_emitente	BIGINT

Staging	Tabela final	Conversao
id_contribuinte_destinatario	id_contribuinte_destinatario	BIGINT
data_emissao	data_emissao	DATE
valor_total	valor_total	DECIMAL(16,2)
situacao	situacao	texto validado pelo CHECK

Carga

```

;WITH dados_validos AS (
    SELECT
        s.chave_acesso,
        TRY_CONVERT(BIGINT, s.id_contribuinte_emitente) AS id_emitente,
        TRY_CONVERT(BIGINT, s.id_contribuinte_destinatario) AS id_destinatario,
        TRY_CONVERT(DATE, s.data_emissao) AS data_emissao,
        TRY_CONVERT(DECIMAL(16,2), s.valor_total) AS valor_total,
        s.situacao
    FROM staging.stg_nfe_bulk AS s
)
INSERT INTO fiscal.NFE (
    chave_acesso, id_contribuinte_emitente, id_contribuinte_destinatario,
    data_emissao, valor_total, situacao
)
SELECT
    d.chave_acesso, d.id_emitente, d.id_destinatario,
    d.data_emissao, d.valor_total, d.situacao
FROM dados_validos AS d
JOIN fiscal.CONTRIBUINTE AS ce ON ce.id_contribuinte = d.id_emitente
JOIN fiscal.CONTRIBUINTE AS cd ON cd.id_contribuinte = d.id_destinatario
WHERE LEN(d.chave_acesso) = 44
    AND d.chave_acesso NOT LIKE '%[^0-9]%'
    AND d.id_emitente IS NOT NULL
    AND d.id_destinatario IS NOT NULL
    AND d.id_emitente <> d.id_destinatario
    AND d.data_emissao IS NOT NULL
    AND d.valor_total IS NOT NULL
    AND d.valor_total >= 0
    AND d.situacao IN ('AUTORIZADA', 'CANCELADA', 'DENEGADA', 'INUTILIZADA')
    AND NOT EXISTS (
        SELECT 1 FROM fiscal.NFE AS n
        WHERE n.chave_acesso = d.chave_acesso
    );

```

O **JOIN** garante que os dois contribuintes existam. O **NOT EXISTS** torna a carga idempotente em relacao a **chave_acesso**.

7.2 ITEM_NFE

```
INSERT INTO fiscal.ITEM_NFE (id_nfe, ncm, cfop, valor_item)
SELECT
    TRY_CONVERT(BIGINT, s.id_nfe),
    s.ncm,
    s.cfop,
    TRY_CONVERT(DECIMAL(16,2), s.valor_item)
FROM staging.stg_item_nfe_bulk AS s
JOIN fiscal.NFE AS n
    ON n.id_nfe = TRY_CONVERT(BIGINT, s.id_nfe)
WHERE TRY_CONVERT(BIGINT, s.id_nfe) IS NOT NULL
    AND LEN(s.ncm) = 8
    AND s.ncm NOT LIKE '%[^0-9]%'
    AND LEN(s.cfop) = 4
    AND s.cfop NOT LIKE '%[^0-9]%'
    AND LEFT(s.cfop, 1) IN ('1', '2', '3', '5', '6', '7')
    AND TRY_CONVERT(DECIMAL(16,2), s.valor_item) >= 0;
```

O **JOIN** impede a insercao de um item sem NF-e pai. Neste material, **id_nfe** do CSV foi preparado para coincidir com o **id_nfe** gerado no exercicio. Em um processo real, o CSV de itens deve trazer uma chave natural da NF-e ou uma tabela de mapeamento, pois valores **IDENTITY** nao devem ser presumidos.

7.3 NFCE

```

INSERT INTO fiscal.NFCE (
    chave_acesso, id_contribuinte_emitente, data_emissao, valor_total, situacao
)
SELECT
    s.chave_acesso,
    TRY_CONVERT(BIGINT, s.id_contribuinte_emitente),
    TRY_CONVERT(DATE, s.data_emissao),
    TRY_CONVERT(DECIMAL(16,2), s.valor_total),
    s.situacao
FROM staging.stg_nfce_bulk AS s
JOIN fiscal.CONTRIBUINTE AS c
    ON c.id_contribuinte = TRY_CONVERT(BIGINT, s.id_contribuinte_emitente)
WHERE LEN(s.chave_acesso) = 44
    AND s.chave_acesso NOT LIKE '%[^0-9]%'
    AND TRY_CONVERT(BIGINT, s.id_contribuinte_emitente) IS NOT NULL
    AND TRY_CONVERT(DATE, s.data_emissao) IS NOT NULL
    AND TRY_CONVERT(DECIMAL(16,2), s.valor_total) >= 0
    AND s.situacao IN ('AUTORIZADA', 'CANCELADA', 'DENEGADA')
    AND NOT EXISTS (
        SELECT 1 FROM fiscal.NFCE AS n
        WHERE n.chave_acesso = s.chave_acesso
    );

```

NFCE possui somente o contribuinte emitente. O consumidor final não é incluído como contribuinte neste modelo.

7.4 ITEM_NFCE

```

INSERT INTO fiscal.ITEM_NFCE (id_nfce, ncm, cfop, valor_item)
SELECT
    TRY_CONVERT(BIGINT, s.id_nfce),
    s.ncm,
    s.cfop,
    TRY_CONVERT(DECIMAL(16,2), s.valor_item)
FROM staging.stg_item_nfce_bulk AS s
JOIN fiscal.NFCE AS n
    ON n.id_nfce = TRY_CONVERT(BIGINT, s.id_nfce)
WHERE TRY_CONVERT(BIGINT, s.id_nfce) IS NOT NULL
    AND LEN(s.ncm) = 8
    AND s.ncm NOT LIKE '%[^0-9]%'
    AND LEN(s.cfop) = 4
    AND s.cfop NOT LIKE '%[^0-9]%'
    AND LEFT(s.cfop, 1) IN ('1', '2', '3', '5', '6', '7')
    AND TRY_CONVERT(DECIMAL(16,2), s.valor_item) >= 0;

```

Assim como em `ITEM_NFE`, o `JOIN` valida a existência da nota pai e a conversão evita que um valor inválido interrompa toda a consulta.

7.5 CTE

```
INSERT INTO fiscal.CTE (  
    chave_acesso, id_contribuinte_emitente, id_nfe_vinculada,  
    valor_frete, situacao  
)  
SELECT  
    s.chave_acesso,  
    TRY_CONVERT(BIGINT, s.id_contribuinte_emitente),  
    TRY_CONVERT(BIGINT, s.id_nfe_vinculada),  
    TRY_CONVERT(DECIMAL(16,2), s.valor_frete),  
    s.situacao  
FROM staging.stg_cte_bulk AS s  
JOIN fiscal.CONTRIBUINTE AS c  
    ON c.id_contribuinte = TRY_CONVERT(BIGINT, s.id_contribuinte_emitente)  
LEFT JOIN fiscal.NFE AS n  
    ON n.id_nfe = TRY_CONVERT(BIGINT, s.id_nfe_vinculada)  
WHERE LEN(s.chave_acesso) = 44  
    AND s.chave_acesso NOT LIKE '%[^0-9]%'  
    AND TRY_CONVERT(BIGINT, s.id_contribuinte_emitente) IS NOT NULL  
    AND TRY_CONVERT(DECIMAL(16,2), s.valor_frete) >= 0  
    AND s.situacao IN ('AUTORIZADO', 'CANCELADO', 'DENEGADO')  
    AND NOT EXISTS (  
        SELECT 1 FROM fiscal.CTE AS x  
        WHERE x.chave_acesso = s.chave_acesso  
    );
```

O **LEFT JOIN** permite que o CT-e seja carregado mesmo quando a NF-e vinculada não foi localizada. O modelo final aceita **id_nfe_vinculada = NULL**.

7.6 EFD_REGISTRO

```
INSERT INTO fiscal.EFD_REGISTRO (  
    id_contribuinte, tipo_registro, periodo_apuracao, id_nfe_referenciada  
)  
SELECT  
    TRY_CONVERT(BIGINT, s.id_contribuinte),  
    s.tipo_registro,  
    s.periodo_apuracao,  
    TRY_CONVERT(BIGINT, s.id_nfe_referenciada)  
FROM staging.stg_efd_registro_bulk AS s  
JOIN fiscal.CONTRIBUINTE AS c  
    ON c.id_contribuinte = TRY_CONVERT(BIGINT, s.id_contribuinte)  
LEFT JOIN fiscal.NFE AS n  
    ON n.id_nfe = TRY_CONVERT(BIGINT, s.id_nfe_referenciada)  
WHERE TRY_CONVERT(BIGINT, s.id_contribuinte) IS NOT NULL  
    AND s.tipo_registro IN ('C100', 'C170')  
    AND s.periodo_apuracao LIKE '[0-1][0-9]/[1-2][0-9][0-9][0-9]';
```

O **LEFT JOIN** mantém o registro EFD quando a NF-e referenciada não existe, porque a FK da tabela final é opcional. O **CHECK** de período valida o formato **MM/YYYY**.

9. Conferencia dos resultados

8.1 Contagens nas tabelas finais

```
SELECT 'CONTRIBUINTE' AS tabela, COUNT(*) AS total FROM fiscal.CONTRIBUINTE
UNION ALL
SELECT 'NFE', COUNT(*) FROM fiscal.NFE
UNION ALL
SELECT 'ITEM_NFE', COUNT(*) FROM fiscal.ITEM_NFE
UNION ALL
SELECT 'NFCE', COUNT(*) FROM fiscal.NFCE
UNION ALL
SELECT 'ITEM_NFCE', COUNT(*) FROM fiscal.ITEM_NFCE
UNION ALL
SELECT 'CTE', COUNT(*) FROM fiscal.CTE
UNION ALL
SELECT 'EFD_REGISTRO', COUNT(*) FROM fiscal.EFD_REGISTRO;
```

8.2 Conferir relacionamentos

```
SELECT i.id_item_nfe
FROM fiscal.ITEM_NFE AS i
LEFT JOIN fiscal.NFE AS n ON n.id_nfe = i.id_nfe
WHERE n.id_nfe IS NULL;

SELECT i.id_item_nfce
FROM fiscal.ITEM_NFCE AS i
LEFT JOIN fiscal.NFCE AS n ON n.id_nfce = i.id_nfce
WHERE n.id_nfce IS NULL;

SELECT c.id_cte
FROM fiscal.CTE AS c
LEFT JOIN fiscal.NFE AS n ON n.id_nfe = c.id_nfe_vinculada
WHERE c.id_nfe_vinculada IS NOT NULL
AND n.id_nfe IS NULL;
```

As três consultas devem retornar zero linhas.

8.3 Conferir situações e valores


```
SELECT situacao, COUNT(*) AS total
FROM fiscal.NFE
GROUP BY situacao;
```

```
SELECT situacao, COUNT(*) AS total
FROM fiscal.NFCE
GROUP BY situacao;
```

```
SELECT situacao, COUNT(*) AS total
FROM fiscal.CTE
GROUP BY situacao;
```

```
SELECT COUNT(*) AS valores_negativos
FROM fiscal.NFE
WHERE valor_total < 0;
```

10. Tratamento de erros e reproprocessamento

9.1 Diferença entre erro de layout e erro de conteúdo

Um arquivo com quantidade incorreta de campos, delimitador errado ou quebra de linha incompatível pode falhar no **BULK INSERT**. Esse é um erro de estrutura do arquivo.

Um arquivo com **valor_total = 'ABC'** pode entrar na staging, pois a coluna é textual. Esse é um erro de conteúdo e deve ser tratado na consolidação com **TRY_CONVERT**.

9.2 Listar linhas rejeitadas antes da carga final

Exemplo para NF-e:

```
SELECT s.*,
CASE
    WHEN LEN(s.chave_aceeso) <> 44
    OR s.chave_aceeso LIKE '%[^0-9]%' THEN 'Chave invalida'
```

```

        WHEN TRY_CONVERT(BIGINT, s.id_contribuinte_emitente) IS NULL
        OR TRY_CONVERT(BIGINT, s.id_contribuinte_destinatario) IS NULL
        THEN 'Contribuinte invalido'
    WHEN TRY_CONVERT(DECIMAL(16,2), s.valor_total) IS NULL
    THEN 'Valor invalido'
    WHEN TRY_CONVERT(DECIMAL(16,2), s.valor_total) < 0
    THEN 'Valor negativo'
    WHEN s.situacao NOT IN ('AUTORIZADA', 'CANCELADA', 'DENEGADA',
'INUTILIZADA')
    THEN 'Situacao invalida'
    ELSE 'Verificar relacionamento ou duplicidade'
END AS motivo
FROM staging.stg_nfe_bulk AS s
WHERE LEN(s.chave_acesso) <> 44
    OR s.chave_acesso LIKE '%[^0-9]%'
    OR TRY_CONVERT(BIGINT, s.id_contribuinte_emitente) IS NULL
    OR TRY_CONVERT(BIGINT, s.id_contribuinte_destinatario) IS NULL
    OR TRY_CONVERT(DECIMAL(16,2), s.valor_total) IS NULL
    OR TRY_CONVERT(DECIMAL(16,2), s.valor_total) < 0
    OR s.situacao NOT IN ('AUTORIZADA', 'CANCELADA', 'DENEGADA', 'INUTILIZADA');

```

Em um processo de producao, o resultado deve ser gravado em uma tabela de rejeicoes com a origem, o motivo, os valores brutos e a data da analise.

9.3 Transacao na consolidacao

Quando varias tabelas finais precisam ser atualizadas como uma unidade, use uma transacao e trate excecoes:

```

BEGIN TRY
    BEGIN TRANSACTION;

    -- INSERTs de consolidacao validados entram aqui.

    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF XACT_STATE() <> 0
        ROLLBACK TRANSACTION;
    THROW;
END CATCH;

```

Os dados da staging nao devem ser apagados antes de conferir as contagens e as rejeicoes.

11. Boas praticas e problemas comuns

ROWTERMINATOR

Se o arquivo for criado no Windows e usar CRLF, teste `ROWTERMINATOR = '0x0A'` ou `ROWTERMINATOR = '\r\n'` conforme o arquivo real. O terminador deve ser confirmado no arquivo, nao escolhido apenas pelo sistema operacional.

Codificacao

Use `CODEPAGE = '65001'` para UTF-8. Se o arquivo vier em outra codificacao, ajuste o parametro depois de confirmar a origem.

Coluna de controle

`imported_dt` nao vem dos CSVs. Por isso, no `criar_stg_nfe_bulk.sql`, ela e adicionada depois da ingestao com `DEFAULT SYSUTCDATETIME()`. Se a tabela ja tiver essa coluna antes do `BULK INSERT`, o mapeamento posicional pode causar erro ou carregar dados no lugar errado.

Chaves IDENTITY

IDs gerados nas tabelas finais podem mudar entre cargas. Para integrar arquivos de entidades relacionadas, prefira chaves naturais ou capture o mapeamento dos IDs gerados. A coincidencia entre os IDs dos arquivos de exemplo e os IDs das tabelas finais e uma simplificacao didatica.

Execucao repetida

Limpe a staging antes de reimportar e verifique se as tabelas finais ja possuem dados. As restricoes `UNIQUE` e as chaves estrangeiras protegem a integridade, mas nao substituem o controle do lote.

MAXERRORS e ERRORFILE

Para arquivos externos, pode-se acrescentar **MAXERRORS** e **ERRORFILE** ao **BULK INSERT**. O arquivo indicado por **ERRORFILE** precisa estar em um caminho gravavel pelo servico do SQL Server e nao pode existir previamente com o mesmo nome.

Exemplo:

```
BULK INSERT staging.stg_nfe_bulk
FROM 'C:\dados\staging\staging_nfe_20.csv'
WITH (
    FIRSTROW = 2,
    FIELDTERMINATOR = ';',
    ROWTERMINATOR = '0x0A',
    CODEPAGE = '65001',
    TABLOCK,
    MAXERRORS = 0,
    ERRORFILE = 'C:\dados\staging\erros_staging_nfe.log'
);
```

Com **MAXERRORS = 0**, a carga para no primeiro erro. Isso e adequado quando nenhum erro de layout pode ser aceito silenciosamente. Para erros de conteudo, mantenha a staging textual e trate-os com consultas de validacao.

Conclusao

O fluxo completo deste material e:

1. disponibilizar os CSVs em um caminho acessivel ao SQL Server;
2. criar o schema e as tabelas de staging;
3. carregar os seis arquivos com **BULK INSERT**;
4. conferir contagens e formatos na staging;
5. criar o schema e as tabelas finais;
6. garantir que os contribuintes existam;
7. inserir NFE e NFCE;
8. inserir itens, CT-e e registros EFD respeitando os relacionamentos;
9. conferir chaves, situacoes, valores e quantidades;
10. preservar e tratar as linhas rejeitadas antes de limpar qualquer staging.

Esse desenho transforma uma carga de arquivos em um processo controlado, auditavel e reprocessavel.